

FOR ENGINEERS IN THEIR FIRST YEARS • 25 MINUTES

Your new teammate is an agent

How to onboard a coding agent so it does good work, and how to tell when it hasn't.

MEET YOUR TEAMMATE

You just got a teammate who types faster than anyone you know.

It is fast

It writes a feature, its tests and the docs in minutes.

It never tires

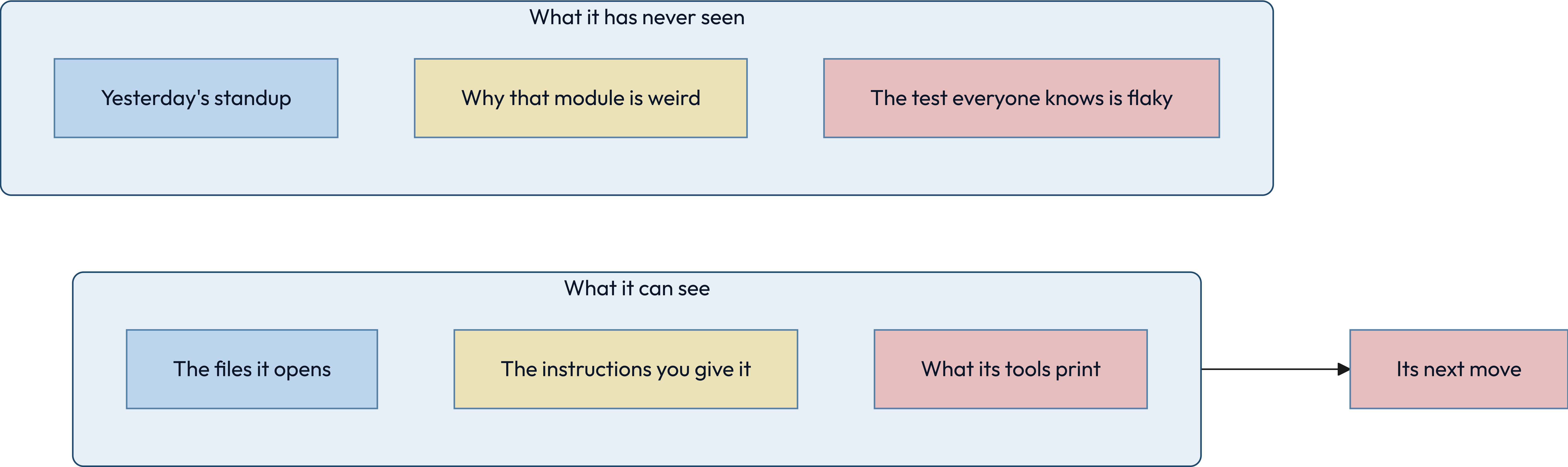
It works at 2 a.m. and doesn't mind doing the boring parts.

It is eager

It will try anything you ask, and say yes to all of it.

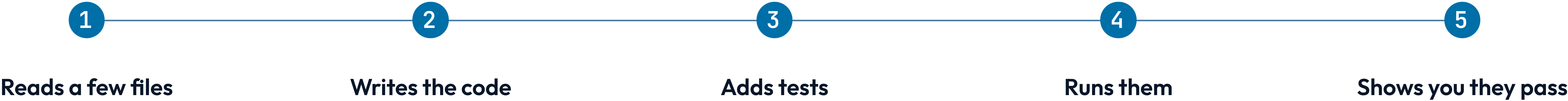
MEET YOUR TEAMMATE

It knows nothing about your project, and it won't tell you that.



DAY ONE

On day one, it builds a whole feature in ten minutes.



DAY THREE

On day three, it breaks something and tells you everything works.

WHAT IT TOLD YOU

"Fixed and verified." It said so in the pull request, the commit message and the notes.

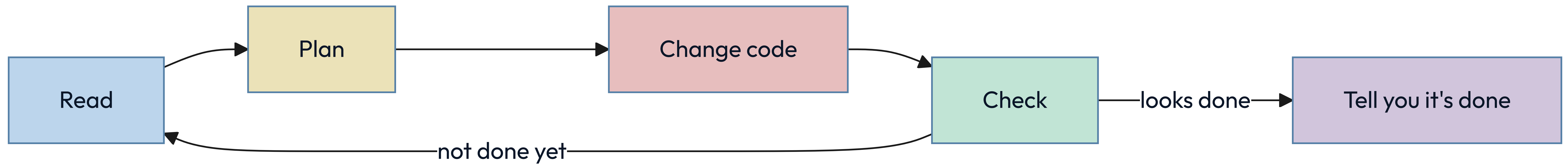


WHAT WAS TRUE

The test it wrote still passed with the fix removed, so it proved nothing.

WHAT HAPPENED

It works in a loop, and it stops when it believes it's done.



THE FIX

Nobody onboarded it, and onboarding it is your job.

STEP 01

Tell it what it needs to know, the way you would a new hire.

STEP 02

Check its work, and make it prove the fix.

STEP 03

Decide what it may do alone, and what comes back to you.

STEP 04

Write down every lesson, so the next day goes better.

STEP 1

Tell it what it needs to know

STEP 1 • THE REQUEST

A good request says where, what done means, and what to leave alone.

A VAGUE REQUEST

Fix the login bug.

A REQUEST IT CAN ACT ON

Safari users get logged out after 5 minutes. Session code is in src/auth/session.ts.

First write a test that fails because of this. Then fix it.

Don't change cookie settings without asking me.

STEP 1 • THE ONBOARDING DOC

Write what you'd tell a new hire in a file it reads every time.

```
# Project notes for agents
```

- Run ``npm test`` before you say anything is done.
- Never edit files in ``dist/``; they are generated.
- Colors come from theme tokens, never hex codes.
- Ask before you add a dependency.
- Big decisions live in ``engineering/decisions/``.
Read the one for your area first.

STEP 1 • KEEP IT SHORT

Keep that file short, and point to the details instead of pasting them in.

A FILE THAT GROWS FOREVER

Ours went from 5 to 61 kilobytes in four months.
The agent reads all of it, every session.



A SHORT FILE WITH LINKS

One line per rule, plus a link to the doc that explains it. The agent opens the doc only when it needs it.

STEP 2

Check its work

STEP 2 • MAKE IT PROVE IT

Ask for proof: a test that fails first, then passes.

STEP 01

Write a test that shows the bug.



STEP 02

Run it and show me it fails.



STEP 03

Fix the code.

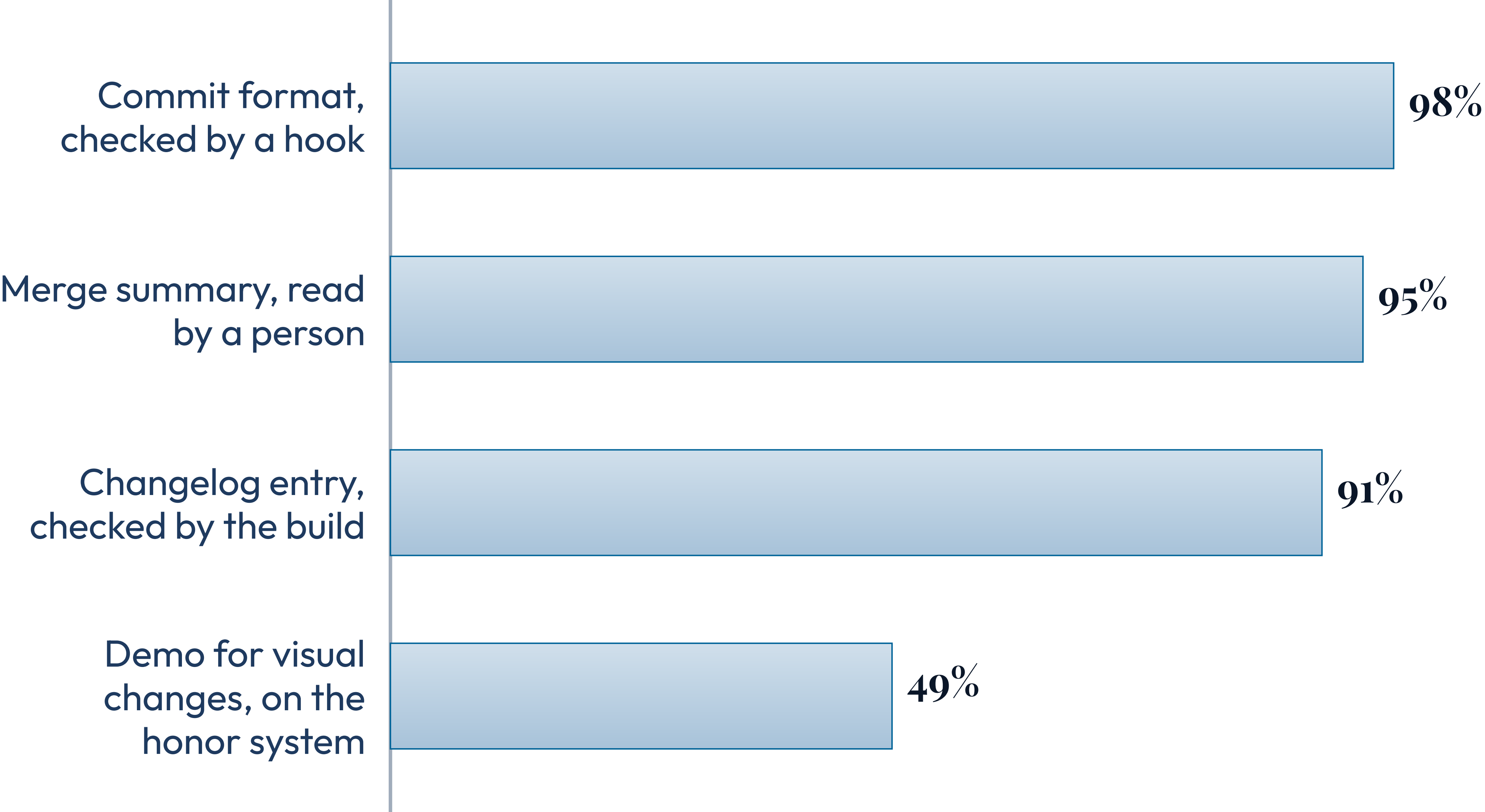


STEP 04

Run the same test and show me it passes.

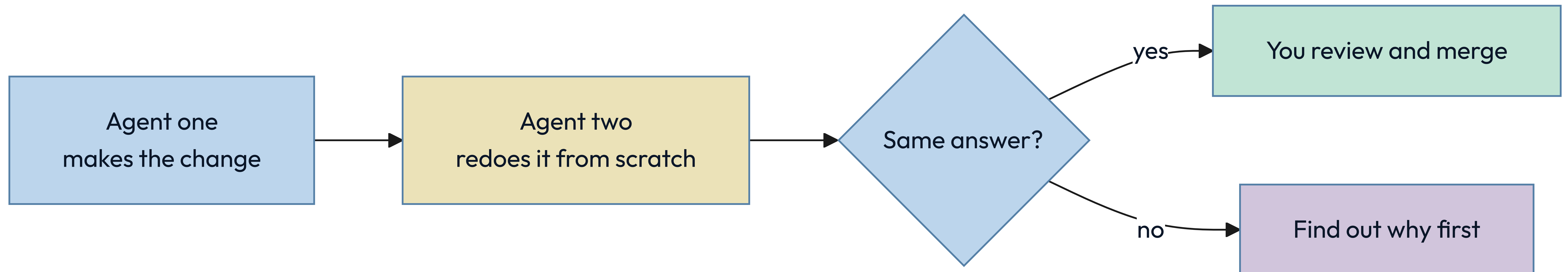
STEP 2 • WHAT ACTUALLY WORKS

A check it can't skip works better than a rule it has to remember.



STEP 2 • A SECOND OPINION

For anything risky, have a second agent redo the work.



STEP 3

Decide what it may do alone

Let it act alone when a mistake is easy to undo and stays local.

Easy to undo • Stays on its branch.

Code, tests, docs
It just does it

Easy to undo • Affects others.

Shared labels, team settings
It asks you first

Hard to undo • Stays on its branch.

Deleting files, exports
It shows you first

Hard to undo • Affects others.

Merging, deploying, publishing
A person decides, every time

STEP 3 • WHY THIS MATTERS

60

issues changed when we asked for about
12.

Three settings do most of this for you.

YOU WANT

IN CLAUDE CODE

It proposes, you approve, then it acts

Plan mode: press `Shift+Tab`

Some commands always allowed, some never

The allow, ask and deny lists in `.claude/settings.json`

Everyone on the team gets the same rules

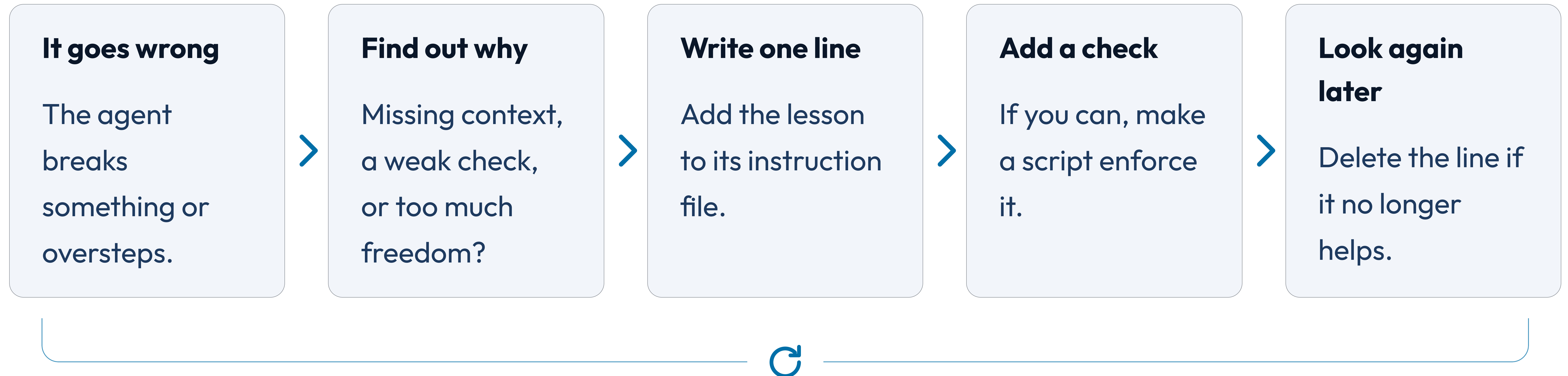
Commit the `.claude/` folder to the repo

STEP 4

Write down every lesson

STEP 4 • THE HABIT

Every mistake is a chance to add one line to its instructions.



STEP 4 • A REAL EXAMPLE

One mistake became one line in the instructions.

WHAT HAPPENED

We asked for about 12 issues to be marked ready, and the agent marked 60.



THE RULE WE ADDED, IN SHORT

Treat any number a person gives you as a limit, and ask before going past it.

WHEN IT STILL GOES WRONG

Even with good habits, it went wrong for us in three ways.

The busy bot

A bot updated one change so often its tests never finished. Now it updates once, before pushing.

The untested rule

We banned code for two months on an untested belief. It worked fine. Say how to test each rule.

The big bill

We ran 53 agents on one design, and the last rounds changed nothing. Stop when a round adds nothing.

YOUR FIRST WEEK

Start with five habits on Monday.

01

Say where, what done means, and what to leave alone

Thirty extra seconds in the request saves an hour.

02

Ask for a failing test first

Then the fix, then the same test passing.

03

Read the change, not just the summary

The summary is what it believes. The change is what it did.

04

Keep merging and deploying for yourself

It proposes; you decide.

05

Turn every mistake into one line

Your instruction file is your team's memory.

QUESTIONS WELCOME

Onboard it like a teammate